

Does the best LLM serving configuration depend on the GPU it runs on?

An experiment report

Ahmad Tawil

ahmadtawil.se@gmail.com

5 August 2026

Abstract

When you run a large language model as a service, you have to choose settings. One of them is whether the server should remember work it has already done, so that repeated parts of a request are not computed twice. Turning that on makes the server faster. The question is by how much, and whether the answer changes if you run the same server on a cheaper graphics card.

I measured it. I ran the same model, the same requests and the same settings on two graphics cards — an expensive A100 and a cheap L4 — and turned that one setting on and off. I did this at eight levels of how busy the server was, from one user at a time up to 128.

Two things came out, and they point in opposite directions.

The setting behaves the same way on both cards. With one user it barely helps: about 1% faster. As the server gets busier it helps more and more, up to 2.56 times faster on the expensive card and 2.70 times on the cheap one. Those numbers are close, and they stay close at every level in between. So if you work out the best setting on one card, that answer carries over to the other card. That is not what I predicted — I predicted the two cards would disagree somewhere — so the prediction I registered in advance is **refuted**.

But the cheap card cannot do the job at all. Before running anything I fixed a speed requirement: each word of the reply should take under 50 milliseconds. The expensive card meets it everywhere. The cheap card never meets it — not at any level of load, not with the setting on or off. It is already too slow with a single user.

So the *advice* transfers between the two cards, but *whether a card can be used at all* does not. How to configure the server is a question you can answer on any card. Whether to buy the cheap card is not.

I also tried to compare two different serving programs, vLLM and TensorRT-LLM. TensorRT-LLM installed but would not start: it needs a graphics library that is not available for the machine it was installed on. I report that as a result rather than leaving it out, because it is a real cost of choosing that program.

1 Introduction

Serving an LLM in production means choosing a configuration: how much of the GPU to give to the key-value cache, how many requests to schedule at once, whether to reuse cached computation across requests that share a prefix. These choices are made once, usually by benchmarking on whatever GPU the team has, and then carried onto whatever hardware turns out to be affordable.

Whether that carry-over is safe has not been systematically tested. If it is not, every published tuning recommendation has an unstated hardware precondition attached to it.

This measurement asks: **does the best vLLM serving configuration depend on the GPU, and does the answer change with how loaded the server is?**

Hypothesis 3, registered before any measurement. From SCOPE.md, committed 5 August 2026:

The optimal vLLM configuration is concurrency-dependent — no single configuration leads across the whole load range — and the concurrency at which the ranking changes depends on

the card, arriving at a different point on the L4 than on the A100, because a card with less memory bandwidth and less KV capacity is rewarded differently by the same setting.

Refuted if one configuration leads at every concurrency level on both cards, **or** the ranking changes at the same concurrency on both.

The refutation condition fired. Section 4 reports it, and reports what the data says instead — which turns out to be a sharper claim than the one I predicted.

1.1 Contribution

1. Four complete concurrency curves ($1 \rightarrow 128$; 3 repeats on the A100, 2 on the L4; 80 records, all successful) for one pinned model on two GPU tiers, with the prefix-caching knob swept and every other input frozen and hashed.
2. A **quantified transfer result**: the payoff from prefix caching as a function of load is nearly identical on two cards differing by $4.35\text{--}5.26\times$ in throughput.
3. A **non-transfer result** from the same records: the L4 fails the registered SLO at every concurrency level under both configurations, while the A100 meets it at every level. Ranking transfers; viability does not.
4. A reported toolchain refusal: TensorRT-LLM 1.2.1 installs but cannot load its own bindings on the target runtime, with the error and the attempted remedies.

2 Related work

Hardware-dependent efficiency is established at the kernel level, and that is deliberately not what this measures. APEX4 [1] is the closest prior work to this programme’s thesis. It identifies the Tensor-Core to CUDA-Core throughput ratio ρ as the primary hardware indicator governing W4A4 quantization efficiency, and reports that the same W4A4-g128 kernel yields $2.0\text{--}2.5\times$ speedup on an RTX 3090 ($\rho = 16$) yet degrades to $0.43\text{--}0.47\times$ on an A100 ($\rho = 64$) in compute-bound scenarios — establishing W4A4 viability as *platform-dependent rather than universally infeasible*. That is the kernel-level version of the question asked here, and it constrains the contribution: that angle is taken.

Two further works occupy the surrounding space. QServe [3] co-designs a W4A8KV4 quantization algorithm with a serving system, and reports $2.4\text{--}3.5\times$ higher throughput than TensorRT-LLM on A100 and L40S. AnyBCQ [5] extends binary-coded quantization to multi-precision inference with direct bit-plane operations, reporting throughput gains of up to $3.0\times$ over half precision and $1.2\times$ over prior multi-precision methods.

Notably, APEX4 deploys its kernels as a drop-in replacement inside *unmodified* vLLM. The framework layer is treated throughout this literature as a fixed substrate rather than as an object of study.

What remains thin is framework- and configuration-level behaviour *under load*. Public comparisons are vendor blog posts reporting a single operating point, rarely cost-normalised and rarely repeated.

Gap. Whether a serving configuration’s benefit changes with concurrency, and whether that behaviour transfers across hardware tiers, is unmeasured.

On the citations. All six references were checked against their primary sources on 5 August 2026 rather than carried forward from the programme’s reference document. That pass found one error: the reference document attributed to AnyBCQ a finding that relative quantization speedups are preserved across A100 and H100. **AnyBCQ makes no such claim**, and an earlier draft of this section repeated it. The description above is what the paper’s abstract actually states. Measurement 01 contained a citation error of the same class — a model cited to its dataset rather than to its architecture — which is why this pass was run at all.

3 Method

3.1 System under test

The system under test is one vLLM server [2] (v0.26.0) behind its OpenAI-compatible completions endpoint. It serves `Qwen/Qwen2.5-7B-Instruct` [6], pinned through `serving/pins.py` to revision `a09a3545...` (full hash in the bibliography) and stamped into every result record.

The model was chosen over an 8B Llama-class alternative for two reasons, both checked rather than assumed. It is ungated, so reproduction does not depend on one account’s licence acceptance. And its grouped-query attention uses 4 KV heads against Llama-3.1-8B’s 8, roughly halving KV-cache footprint per token — which is what makes concurrency 128 fit on a 24 GB L4 at all. The arithmetic sits in `serving/config.py` beside the constants it justifies.

The model is natively bf16 and is served in fp16 (`-dtype float16`). vLLM logs this cast explicitly; it is a scope decision, not an accident.

3.2 The swept configuration

Axis	Levels
Prefix caching	on / off
Card	A100-SXM4-40GB · L4 24 GB
Concurrency	1 · 2 · 4 · 8 · 16 · 32 · 64 · 128

Four curves, eight points each. Three repeats per point on the A100, two on the L4 (Section 3.7).

Held fixed, and verified identical across all 80 records by `analysis.py`’s integrity check rather than assumed: model; model revision `a09a3545...`; prompt set `prompts_sha 3b3efb02...`; 512 input tokens; 128 output tokens forced with `min_tokens`; max model length 2048; greedy sampling at `temperature=0`; fp16.

Forcing the output length matters more than it looks. If one configuration produced shorter replies, its time-per-output-token would improve for free and the comparison would be void.

Two vLLM defaults were left at their defaults and are reported as configuration rather than swept: chunked prefill (on) and asynchronous scheduling (on). Both affect the shape of a concurrency curve. Both are constant across all four curves, so neither can explain any difference reported here — but they do mean the absolute numbers describe vLLM at its defaults, not a tuned server.

3.3 Workload

Eight prompts, each padded to exactly 512 tokens under the pinned model’s own tokenizer, frozen to `data/prompts.json` and hashed.

The prompt set is small, and that is load-bearing for reading the caching-on curves. A 60-second window at concurrency 128 issues roughly 2,900 requests, so each of the eight prompts is reused hundreds of times and the measured prefix-cache hit rate reaches **96.9%**. The caching-on curves are therefore an *upper bound* — the ceiling under maximal prompt reuse, not a typical production workload. A workload with diverse prompts would sit between the two curves. Section 5 returns to this.

This was not the original design. The first A100 curve was collected with vLLM’s default (caching on), and the 96.9% hit rate was discovered afterwards from the server’s own log — at which point the card was found to be running at 5.2% KV-cache utilisation with zero queue depth at concurrency 128, i.e. never loaded. Caching-off curves were added, the knob became the study’s axis, and both regimes are reported.

3.4 Harness and measurement

Closed-loop load. N asynchronous workers, each holding exactly one request in flight: send, await the full streamed response, send the next. In-flight count is exactly N at all times, which is what “concurrency N ” means throughout. Open-loop (fixed arrival rate) is the other valid design but produces queue collapse above saturation, which would bury configuration differences in queueing delay.

Latency is measured client-side, on the stream. Three timestamps are taken per request, all in the client:

t_0	the request is sent
t_1	the first streamed chunk arrives
t_2	the last streamed chunk arrives

From those, with n the number of output tokens in the response:

$$\text{TTFT} = t_1 - t_0 \tag{1}$$

$$\text{TPOT} = \frac{t_2 - t_1}{n - 1} \tag{2}$$

TTFT is the wait before any text appears. TPOT is the mean interval between consecutive output tokens thereafter.

The denominator is $n - 1$, not n . The first token’s arrival is already charged to TTFT; only the remaining $n - 1$ tokens arrive during the interval $t_2 - t_1$, so there are $n - 1$ gaps to average over. Dividing by n instead would understate TPOT — and it would understate it equally for every configuration, which is the kind of error that survives review because it cancels out of every ratio in Section 4.

Warmup. 32 discarded requests at the target concurrency — not at concurrency 1 — before a fixed 60-second measurement window. First requests trigger CUDA graph capture and lazy kernel compilation; those are engine artifacts, not steady state.

The load generator was validated before any real number was trusted. A client unable to sustain 128 concurrent streams would produce a flat curve that looks like server saturation. The full sweep was therefore run against a mock server returning a canned stream on a fixed sleep. Throughput tracked linearly with concurrency across the whole range (ratio to linear 0.93–1.06, no downward trend at the top of the axis) and client CPU never exceeded 51% mean. Under real load client CPU peaked at 22.8%.

One record per repeat. Records are aggregated at read time, so run-to-run spread stays recoverable. Every record carries GPU, driver, CUDA version, torch version, git SHA, model revision and `prompts_sha`.

3.5 Hardware and software

Colab Pro. **A100-SXM4-40GB** and **L4 24 GB**, driver 580.82.07, host CUDA 13.0, torch 2.11.0+cu128, vllm 0.26.0.

Measured KV-cache capacity, as reported by vLLM at startup:

Card	KV cache	Max concurrent 2048-token requests
A100-SXM4-40GB	391,152 tokens	190.99×
L4 24 GB	91,264 tokens	44.56×

The L4 was swept to concurrency 128 against a capacity of 44.56 — roughly 2.9× oversubscribed at the top of the axis. That is deliberate, and it is where the L4’s 16.9-second TTFT p95 at concurrency 128 comes from.

Installation was not free, and the friction is reportable. vLLM’s PyPI wheel ships CUDA 13 binaries while the runtime’s torch was a CUDA 12.8 build; `import vllm` failed with `ImportError: libcudart.so.13`. The library was present on disk (`nvidia-cuda-runtime 13.3.29`) but its directory was not registered with the dynamic loader. Registering it with `ldconfig` fixed the import without reinstalling anything. Framework installation being coupled to a CUDA version the user does not control is a real constraint on rented infrastructure, and it is absent from the published framework comparisons.

3.6 Metric definitions

Metric	Definition
Output tokens/sec	total output tokens ÷ measurement window, all concurrent requests
TTFT p50 / p95	client-side, request sent → first streamed chunk
TPOT p50 / p95	client-side, (last – first chunk) ÷ (output tokens – 1)
Goodput	requests/sec completing under the SLO
Tokens per dollar-hour	output tokens/sec ÷ host hourly price

SLO: TTFT < 1000 ms and TPOT < 50 ms. Both thresholds were fixed on the first day of the block, before any measurement, and are not revised. Section 5 reports how the conclusion moves if the TPOT threshold is relaxed.

Percentiles, never means: TTFT and TPOT distributions are right-skewed, and a mean TTFT is a misleading number.

3.7 Reproduction

```
python scripts/serve_vllm.py --card A100-SXM4-40GB --precision fp16
# add --prefix-caching for the ON curve
python scripts/sweep.py --base-url http://localhost:8000 \
  --framework vllm --card A100-SXM4-40GB --precision fp16 \
  --window-s 60 --repeats 3 --warmup-requests 32 --out results/<name>.jsonl
python scripts/analysis.py --emit markdown
python scripts/plot_results.py
```

Every number in Section 4 is printed by analysis.py from the result files. No figure in this paper is typed by hand. This is a direct response to a verification pass which found three numbers in the working notes stated more confidently than the data supported: a run-to-run spread quoted for one curve and applied to four, a throughput ratio quoted as a single value when it varied across operating points, and a “triples” that was $2.95\times$. None changed a conclusion; all three were prose drifting from the data it described. The remedy is mechanical rather than a resolution to be more careful.

Repeat counts differ by card — 3 on the A100, 2 on the L4. The reduction was a deliberate budget decision taken after the A100 curves showed run-to-run spread of at most 1.66% anywhere and 0.004% at concurrency 1. Stated rather than hidden.

The four curves were collected at four different git SHAs (16f2987, fcd053b, 562a207, e3dea24) because code changed between sessions. The diffs were checked: only log files, scope files, result files and the two server-launch scripts changed. `client.py`, `sweep.py` and `record.py` — the entire measurement path — were untouched across all four curves.

3.8 What was attempted and abandoned

The study was originally designed as a **framework** comparison: vLLM against TensorRT-LLM [4], across the same two cards and the same concurrency axis. That axis was dropped, and dropping it was governed by a rule written before it became inconvenient.

SCOPE.md, committed on the block’s first day, specified a hard stop: if neither card’s TensorRT-LLM engine was serving by the end of the fourth day, the framework axis would be dropped entirely and a

named contingency — the vLLM configuration axis — would run instead. A 30-minute timeboxed spike was scheduled on the first day precisely so the answer would be cheap.

The spike failed, inside its budget. TensorRT-LLM 1.2.1 installed successfully in 266 seconds, downgrading torch to 2.9.1 in the process, and then:

```
File "tensorrt_llm/_utils.py", line 47, in <module>
    from tensorrt_llm.bindings import DataType, GptJsonConfig, LayerType
ImportError: libcublasLt.so.13: cannot open shared object file:
    No such file or directory
```

Three remedies were attempted and all failed: registering every NVIDIA library directory with `ldconfig` (the library is genuinely absent, not misplaced); `pip install nvidia-cublas-cu13` from PyPI (Failed building wheel — the entry has no binary); and the same from NVIDIA’s own index, with identical failure. No engine build was attempted on either card, so **no build-time data exists and none is claimed**.

Two things here are results rather than mishaps:

1. **Both frameworks shipped CUDA 13 binaries onto a CUDA 12.8 host and both failed at the dynamic loader.** vLLM’s missing dependency was present and merely unregistered — recoverable in one line. TensorRT-LLM’s was absent with no installable source. Same failure mode, opposite outcomes.
2. **TensorRT-LLM 1.2.1 pins torch 2.9.1; vLLM 0.26.0 requires torch 2.11.0.** The two frameworks cannot share an environment. Running both means two installations, two CUDA stacks and two instances of this class of problem — an operational cost that framework comparisons do not report.

The thesis question is unaffected by the swap, because it was never *which framework wins*; it is whether the answer depends on the hardware. A configuration axis tests that as directly.

4 Results

All numbers below are `analysis.py` output. 80 records, all `status: ok`, controls verified identical across all four curves.

4.1 Throughput, and the payoff from prefix caching

Table 1: Output tokens/sec, and the ratio of caching-on to caching-off. Worst run-to-run spread anywhere in the grid: 1.66% (A100 caching-on, concurrency 16). At concurrency 1 the spreads are 0.004% (A100 off), 0.004% (A100 on), 0.005% (L4 off), 0.001% (L4 on).

Conc.	A100 off	A100 on	L4 off	L4 on	A100 on/off	L4 on/off
1	81.3	82.4	17.4	17.6	1.01×	1.01×
2	159.9	165.0	33.4	34.4	1.03×	1.03×
4	309.1	330.7	64.2	68.2	1.07×	1.06×
8	567.7	648.7	118.0	133.5	1.14×	1.13×
16	981.2	1257.2	197.9	259.8	1.28×	1.31×
32	1519.1	2329.2	318.6	443.0	1.53×	1.39×
64	2111.3	4098.9	452.1	830.4	1.94×	1.84×
128	2407.1	6158.4	524.2	1414.2	2.56×	2.70×

4.2 Verdict: Hypothesis 3 is refuted

Prefix caching leads at **every** concurrency level on **both** cards — 16 comparisons, zero counter-examples. There is no ranking change, and therefore no ranking-change point that could move with the card. The first clause of the registered refutation condition fires.

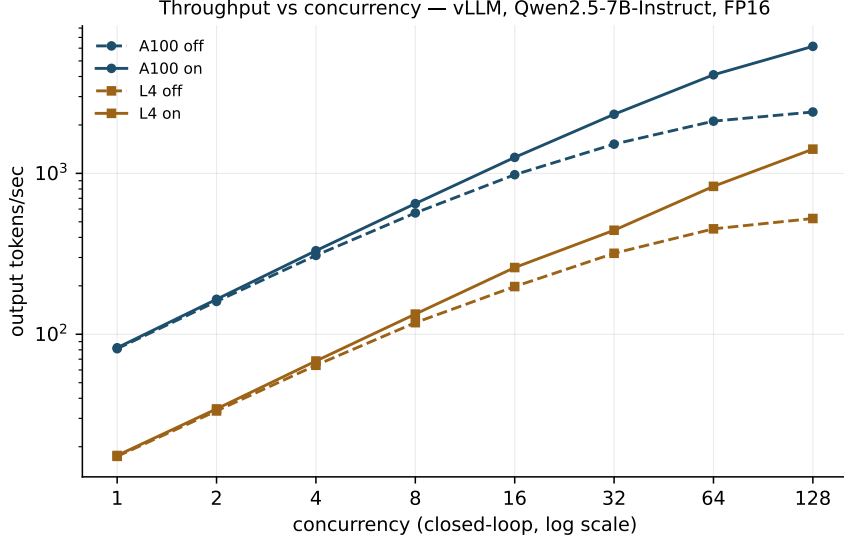


Figure 1: Throughput against concurrency, all four curves. Solid = prefix caching on, dashed = off. Blue = A100, amber = L4.

The margin at concurrency 1 is small — a 1.37% gain on the A100 and 1.25% on the L4 — but repeat spread at that concurrency is 0.004% and 0.005%, so the gain is two orders of magnitude above the noise. It is a real if trivial win, reported as such rather than rounded into a tie to manufacture a crossover.

4.3 The payoff transfers across hardware tiers

Table 2: The two ratio columns of Table 1, read side by side.

Conc.	1	2	4	8	16	32	64	128
A100	1.01×	1.03×	1.07×	1.14×	1.28×	1.53×	1.94×	2.56×
L4	1.01×	1.03×	1.06×	1.13×	1.31×	1.39×	1.84×	2.70×

Same shape, same monotonic climb, comparable magnitudes at every level — across cards differing by 4.35–5.26× in raw throughput (4.67× at concurrency 1) and 4.3× in KV-cache capacity.

The practical reading: *a practitioner who determines prefix-caching policy on an A100 gets the right answer for an L4*. This agrees with Measurement 01’s finding that most of the latency ranking transfers across the same two cards (Kendall $\tau = 0.80$), and the two measurements are independent.

4.4 But viability does not transfer at all

Table 3: TPOT p50 (ms) and goodput (req/s) under the registered SLO (TTFT < 1000 ms and TPOT < 50 ms).

Conc.	TPOT p50 (ms)				Goodput (req/s)			
	A100 off	A100 on	L4 off	L4 on	A100 off	A100 on	L4 off	L4 on
1	12.07	12.06	56.56	56.61	0.64	0.64	0.00	0.00
2	12.10	11.97	58.13	57.68	1.25	1.29	0.00	0.00
4	12.03	11.95	58.03	58.10	2.41	2.58	0.00	0.00
8	12.63	12.12	60.88	58.92	4.43	5.07	0.00	0.00
16	14.09	12.42	67.71	60.37	7.67	9.82	0.00	0.00
32	18.84	13.43	87.06	65.04	11.80	18.20	0.00	0.00
64	27.36	15.18	125.30	74.37	15.91	32.02	0.00	0.00
128	49.08	19.88	204.57	86.51	16.30	48.11	0.00	0.00

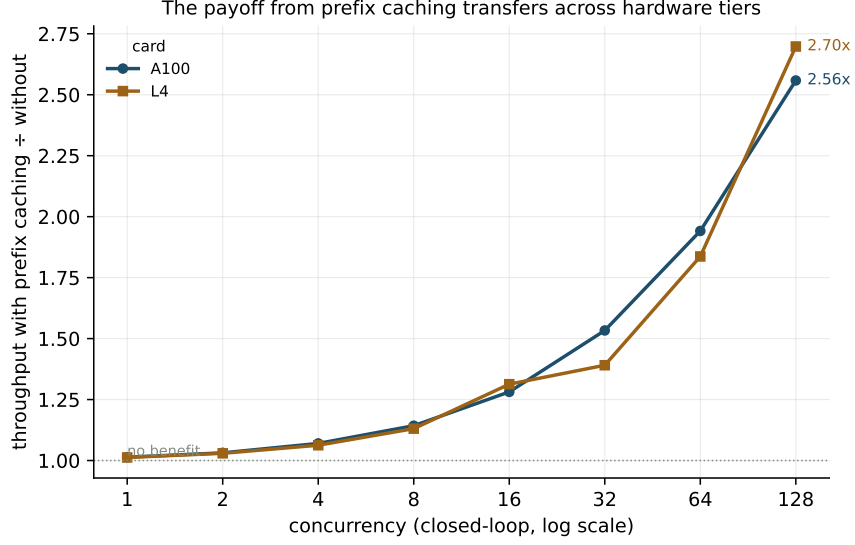


Figure 2: The headline result. Prefix-caching payoff against concurrency, per card. The two lines lie almost on top of each other across a 4.35–5.26× hardware gap. No price assumption and no derived quantity enters this figure.

The L4 serves zero SLO-compliant requests at every concurrency level under both configurations. TPOT p50 is 56.56 ms at concurrency 1 — past the 50 ms limit before any load arrives. Caching improves it substantially at high load (86.51 ms against 204.57 ms at concurrency 128) but never brings it under the threshold.

On the A100 the same knob is worth 2.95× in goodput at concurrency 128 (16.30 → 48.11), because it holds TPOT at 19.88 ms where the uncached configuration reaches 49.08 ms and begins to fail.

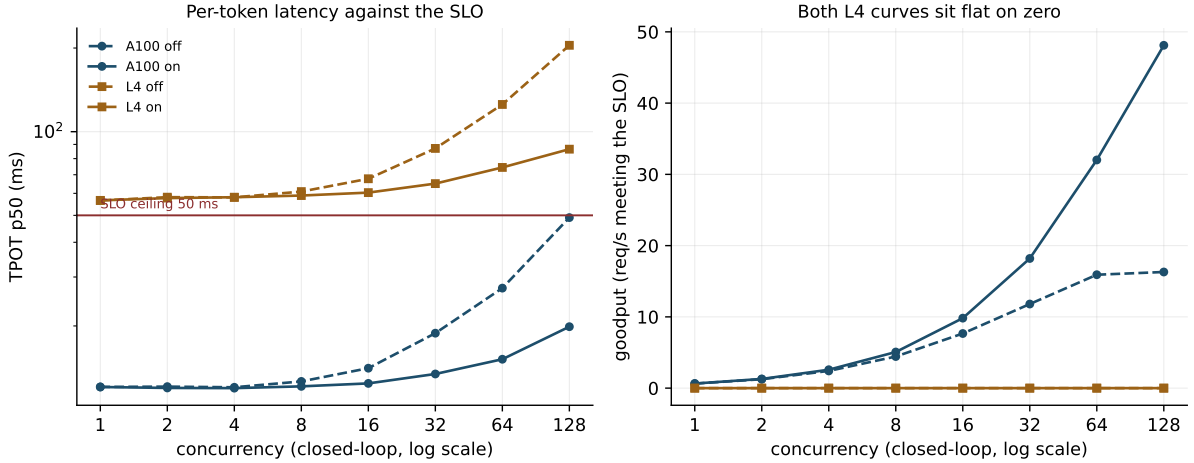


Figure 3: Left: per-token latency against the SLO ceiling. Right: goodput. Both L4 curves sit flat on zero at every concurrency level.

So the same setting plays a different *kind* of role on each card. On the A100 it is a throughput optimisation. On the L4 it more than halves per-token latency and still leaves the card unusable under this SLO.

4.5 Cost, and why the cost claim is weak

At these prices the L4 delivers 6–20% more tokens per dollar at every level. **That ordering is not robust.**

The assumed A100 price of \$3.67/hr sits barely above that range. A $\pm 10\%$ price swing in the

Table 4: Cost-normalised throughput (output tokens/sec per dollar-hour) at GCP list on-demand prices of \$3.67/hr (A100 40 GB) and \$0.70/hr (L4). **These prices are not measured** — see Section 5.

Conc.	A100 off	L4 off	L4/A100	A100 on	L4 on	L4/A100
1	22.2	24.9	1.12×	22.5	25.2	1.12×
2	43.6	47.7	1.09×	45.0	49.1	1.09×
4	84.2	91.7	1.09×	90.1	97.4	1.08×
8	154.7	168.6	1.09×	176.8	190.7	1.08×
16	267.4	282.7	1.06×	342.6	371.2	1.08×
32	413.9	455.1	1.10×	634.7	632.9	1.00×
64	575.3	645.8	1.12×	1116.9	1186.3	1.06×
128	655.9	748.9	1.14×	1678.0	2020.3	1.20×

Table 5: Break-even A100 price, holding the L4 at \$0.70/hr. Above it the L4 is cheaper per token; below it the A100 is.

Configuration	Break-even A100 price
caching off	\$3.21 – \$3.47 /hr
caching on	\$3.05 – \$3.68 /hr

unfavourable direction reverses the ordering (worst-case minimum ratio $0.87\times$ caching-off, $0.82\times$ caching-on); even $\pm 5\%$ reverses it at some levels.

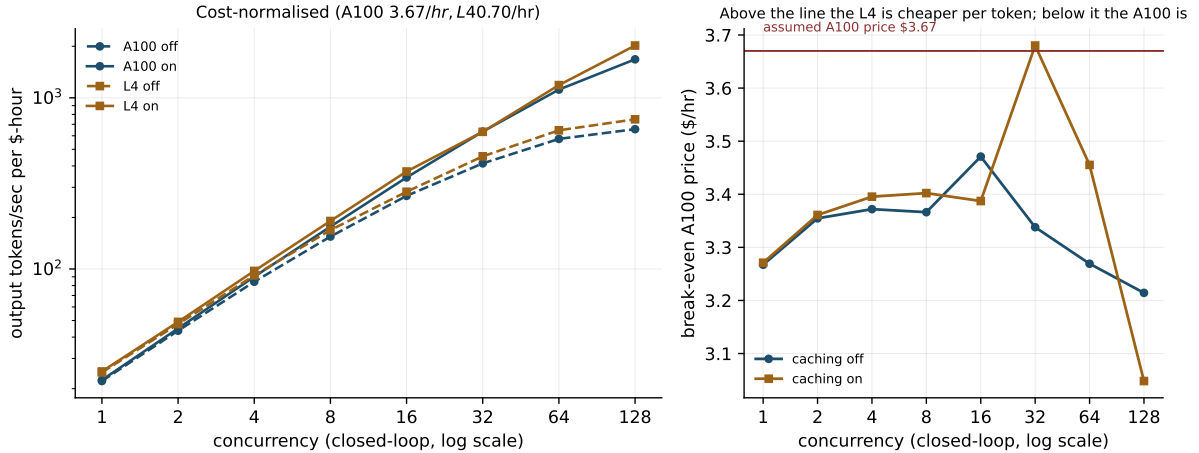


Figure 4: Left: cost-normalised throughput. Right: the break-even A100 price at each concurrency level, against the assumed price, so a reader can check it against their own contract rather than inheriting mine.

The honest statement is therefore:

At GCP list on-demand prices the L4 delivers 6–20% more tokens per dollar, but the break-even A100 price is \$3.05–\$3.68/hr and the ordering does not survive a $\pm 10\%$ price swing. The cost comparison is too close to call without contract prices.

5 Threats to validity

Prices are secondary-sourced. The \$3.67 and \$0.70 figures come from third-party summaries of GCP list pricing, not from Google’s own page, which lists the A100 40 GB only as part of `a2-highgpu` machine types rather than as a standalone GPU line item. Every cost conclusion in Section 4.5 inherits that uncertainty, which is why the break-even and the sensitivity analysis are reported alongside the ratio rather than instead of it.

The prompt set is eight prompts, and the caching-on curves are a ceiling. The measured 96.9% prefix-cache hit rate is an upper bound on realistic reuse. A production workload with diverse prompts would sit between the two curves. The transfer result is a statement about how the *ceiling* behaves on two cards, not about a typical workload.

Table 6: Median-request compliance across TPOT budgets. Computed from stored p50 latencies, so this reports whether the *median* request complies, not true goodput at each threshold — the records do not retain per-request latencies.

TPOT budget	A100 off	A100 on	L4 off	L4 on
50 ms	all levels	all levels	never	never
75 ms	all	all	never	up to 64
100 ms	all	all	up to 8	all levels

The SLO is a working assumption, and the L4 conclusion is sensitive to it. At a 100 ms budget the L4 becomes viable at every concurrency — **but only with caching on**. Without it the L4 fails above concurrency 8. That is a stronger form of Section 4.4’s claim: at some latency budgets the configuration is not an optimisation on the L4 but a precondition for using the card at all.

One curve’s configuration is not confirmed by the server’s own log. The A100 caching-off run’s log was not saved, so `enable_prefix_caching=False` for that curve rests on the launcher’s default at that git SHA rather than on vLLM’s startup banner, which is available for the other three. The behavioural evidence is strong — the curve differs from the caching-on curve by $2.56\times$ at concurrency 128 — but it is inference.

This matters because a closely related failure did occur. During the L4 pass a server was launched onto an occupied port, died with `EADDRINUSE`, and the health check was then answered by the *previous* server; the resulting file was a valid-looking duplicate of the run it was meant to be compared against. It was caught only because the on/off ratio came out at exactly $1.00\times$ at all eight levels — too clean to be physical. `serve_common.py` now refuses to launch onto an occupied port and fails if the launched process exits, but the A100 curve predates that guard.

Single provider, shared hardware. All runs are on Colab Pro, where hardware is shared and the exact host configuration is not under the user’s control. Absolute throughput figures should be read as indicative. The ratios — which are the contribution — are internally consistent, because both members of each ratio were measured in the same session on the same allocation.

Two vLLM defaults were not swept. Chunked prefill and asynchronous scheduling were left on. Both affect a concurrency curve’s shape. They are constant across all four curves so they cannot explain any reported difference, but the absolute numbers describe vLLM at its defaults.

One model, one size, one sequence-length pair. 512 input / 128 output tokens sets the prefill-to-decode balance, and a different pair would move the numbers. Not swept; stated as a scope limit.

Repeat counts differ across cards — 3 on the A100, 2 on the L4 — as a deliberate budget decision taken after observing spread of at most 1.66%.

Missing instrumentation. `kv_cache_util_pct`, `peak_vram_mb`, `gpu_util_pct` and `framework_version` are null in every record. KV-cache utilisation is precisely the diagnostic that would have surfaced the Section 3.3 prefix-caching problem automatically, instead of requiring the server’s stdout to be read by hand.

The programme this belongs to has two completed measurements, not three. Measurement 02 (video ingestion) was scoped, tooled, and halted at its annotation step; its log records why. Measurement

03 was chosen to follow it specifically because its ground truth is machine-generated and the failure that stopped Measurement 02 could not recur.

6 Conclusion

I set out to find a point where the best serving configuration changes with load, and to show that the point moves with the GPU. Neither happened. Prefix caching wins at every concurrency level on both cards, so the pre-registered hypothesis is refuted on its own terms.

What the data shows instead is a cleaner distinction than the one I predicted. The *benefit* of the configuration transfers almost exactly across a $4.35\text{--}5.26\times$ hardware gap: $1.01\times$ at concurrency 1 rising to $2.56\times$ on the A100 and $2.70\times$ on the L4, tracking each other at every level in between. But the *usability* of the hardware does not transfer at all: under the SLO fixed before measurement, the A100 complies at every concurrency and the L4 at none.

The relative recommendation transfers; the absolute viability does not.

For a practitioner that separates two questions usually asked together. *How should I configure the server?* can be answered on whatever card is available. *Can this card serve my workload?* cannot be answered anywhere but on the card itself.

The cost comparison, intended to be the headline, turns out to be the weakest claim in the paper. At list prices the cheap card leads on tokens per dollar by 6–20%, but the break-even is close enough that a $\pm 10\%$ price movement reverses it. It is reported with its break-even rather than as a ratio.

Scope. One model, one provider, two cards, one load pattern, one prompt-reuse regime, and a framework comparison that was attempted and abandoned for a documented toolchain reason. The transfer result rests on 80 records with run-to-run spread of at most 1.66%. The cost result rests on prices I did not verify.

Artefacts

- Result records: `results/vllm_{a100,l4}_fp16_*.jsonl` — 80 records, all ok
- Harness validation: `results/harness_validation.md`
- Analysis: `scripts/analysis.py` — produces every number in Section 4
- Figures: `scripts/plot_results.py`
- Pre-registration: `SCOPE.md`
- Full record of what happened, including what went wrong: `LOG.md`

References

- [1] Hong Guo, Nianhui Guo, Weixing Wang, Jona Otholt, Christoph Meinel, and Haojin Yang. APEX4: Efficient pure W4A4 LLM inference via intra-SM compute rebalancing. *arXiv preprint arXiv:2606.08761*, 2026. Hasso Plattner Institute; Green Bit.AI; German University of Digital Science. Verified 5 Aug 2026.
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180. Verified 5 Aug 2026.
- [3] Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. QServe: W4A8KV4 quantization and system co-design for efficient LLM serving. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025. arXiv:2405.04532. MIT; NVIDIA; MIT-IBM Watson AI Lab; UMass Amherst. Verified 5 Aug 2026.

- [4] NVIDIA. TensorRT-LLM, 2026. Version 1.2.1, as installed on 5 Aug 2026. Cited for the toolchain refusal in Section 3.8 only.
- [5] Gunho Park, Jeongin Bae, Beomseok Kwon, Byeongwook Kim, Se Jung Kwon, and Dongsoo Lee. AnyBCQ: Hardware efficient flexible binary-coded quantization for multi-precision LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.10467. Verified 5 Aug 2026. NOTE: this paper does not claim that relative quantization speedups are preserved across A100 and H100; that attribution was an error in the programme’s reference document and has been corrected.
- [6] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. Model Qwen/Qwen2.5-7B-Instruct, revision a09a35458c702b33eeacc393d103063234e8bc28. Verified 5 Aug 2026.